Republic of Tunisia
Ministry of Higher Education
and Scientific Research
University of Carthage

Scope Statement

SmartCare
IoT-Based Patient Monitoring & Early Warning System

Authored By :

GAIDA Nour
KAROUS Hadil
SHILLI Nour

Supervised By :

Mohamed-Bécha Kaâniche

University Year : **2026–2027**

Higher School of Communications of Tunis
SUP'COM
2083 Communications Technology City - Elghazala - Ariana - Tunisia
Tel. +216 70 240 900 – website: www.supcom.tn

SUP'COM
المدرسة العليا للمواصلات بتونس
École Supérieure des Communications de Tunis
Higher School of Communications of Tunis

Table of Contents

Acronyms iii

Introduction 1

1 Concept and Justification 2
1.1 Context and Problem Statement . . . 2
1.2 Proposed Solution . . . 2

2 Customers, Needs and Objectives 3
2.1 Users and Customers . . . 3
2.2 Usage Scenarios . . . 3
2.3 Measurable Objectives . . . 4
2.4 Project Limits and Priorities . . . 4

3 Functional Scope 5
3.1 Front-Office PWA for the Monitored Individual . . . 5
3.2 Back-Office PWA for Professionals and Administrators . . . 5
3.3 Shared Functions and Alert Lifecycle . . . 6
3.4 Interface Boundaries . . . 6

4 Business Model 7
4.1 Value Proposition and Customers . . . 7
4.2 Revenue and Market Validation . . . 7
4.3 Marketing Mix and Commercialization . . . 7
4.4 Business Model Canvas . . . 8

5 Technical Architecture and Deployment 9
5.1 Overview and Technology Choices . . . 9
5.2 Data Flow and Communication Contracts . . . 10
5.3 Separate PWAs, Security and Reliability . . . 11
5.4 Cloud Deployment and UML Deployment Diagram . . . 11
5.5 Machine Learning Integration and Model Development . . . 12
5.6 MLOps Implementation . . . 14
5.7 Standard IoT Models, WoT and Design Patterns . . . 15

6 Deliverables and Validation Criteria 17
6.1 Deliverables of the Scope Statement Phase . . . 17
6.2 Deliverables of the Development Phase . . . 17
6.3 Validation and Traceability . . . 18

7 Constraints and Methodology 19
7.1 Technological and Hardware Constraints . . . 19
7.2 Time, Budget and Scope Control . . . 19
7.3 Security, Privacy and Retention . . . 19
7.4 eXtreme Programming Methodology . . . 20
7.5 Risks and Preventive Measures . . . 20

8 Planning and Effort Estimation 21
8.1 Planning Assumptions . . . 21
8.2 Tasks, Dependencies, and Effort . . . 21
8.3 Milestones . . . 22
8.4 Gantt Chart . . . 22
8.5 Effort Estimation Method . . . 23

Conclusion 24

Bibliography 25

List of Figures

3.1 Alert episode lifecycle 6

4.1 SmartCare Business Model Canvas 8

5.1 Functional architecture of SmartCare 9
5.2 UML deployment diagram 12
5.3 Machine learning pipeline 14
5.4 MLOps lifecycle and controlled promotion 15
5.5 WoT interpretation for SmartCare 16

7.1 Weekly XP iteration 20

8.1 Six-week project Gantt chart 23

Acronyms

ACL	Access Control List
API	Application Programming Interface
B2B2C	Business-to-Business-to-Consumer
BMC	Business Model Canvas
CI	Continuous Integration
CI/CD	Continuous Integration / Continuous Delivery
CoT	Cloud of Things
CPU	Central Processing Unit
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DB	Database
DVC	Data Version Control
ECG	Electrocardiogram
EE	Enterprise Edition (Jakarta EE)
F1	F1 score: harmonic mean of precision and recall
GB	Gigabyte
GPU	Graphics Processing Unit
HR	Heart rate
HRV	Heart Rate Variability
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaaS	Infrastructure as a Service
ID	Identifier
IoT	Internet of Things
IQR	Interquartile Range
JSON	JavaScript Object Notation
MAD	Median Absolute Deviation
ML	Machine Learning
MLOps	Machine Learning Operations
MQTT	Message Queuing Telemetry Transport
PC	Personal Computer
PDF	Portable Document Format
PR-AUC	Area Under the Precision–Recall Curve
PSI	Population Stability Index
PWA	Progressive Web Application
RAM	Random Access Memory
REST	Representational State Transfer
SpO_2	Estimated peripheral oxygen saturation
TLS	Transport Layer Security
TND	Tunisian dinar
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
USB	Universal Serial Bus
UTC	Coordinated Universal Time
vCPU	Virtual Central Processing Unit
VM	Virtual Machine
W3C	World Wide Web Consortium
WAR	Web Application Archive
WoT	Web of Things
WS	WebSocket
WSS	WebSocket Secure
XP	eXtreme Programming

Introduction

SmartCare is a Cloud of Things prototype for collecting physiological measurements, following their evolution and reporting persistent unusual variations. It combines an ESP32 and MAX30102 acquisition device, MQTT transport, a Node-RED integration gateway, a Jakarta EE 11 backend, MongoDB storage and a Python analysis service. Two separate installable PWAs serve the monitored individual and the supervision team.

This scope statement defines the concept, customers, functions, business model, technical architecture, deliverables, constraints and planning. It specifies the Isolation Forest experiment and the MLOps workflow needed to reproduce, evaluate, deploy, monitor and restore model versions. The Business Model Canvas and deployment figures summarize the business and technical choices.

The document concerns an educational engineering prototype. Estimated physiological values and anomaly scores do not establish a medical diagnosis. Clinical validation, treatment decisions and emergency intervention are excluded. Numerical performance targets are proposed laboratory acceptance criteria, not results already achieved. The six-week development calendar remains subject to confirmation against the course schedule.

Chapter 1

Concept and Justification

Introduction

This chapter introduces the monitoring problem and SmartCare's intended value.

1.1 Context and Problem Statement

Isolated measurements make trends and remote supervision difficult. SmartCare addresses the need for a consistent history with understandable quality information. The question is how to collect timestamped physiological data, reject unreliable estimates and report persistent deviations to authorized users, while tracing model behavior to reproducible experiments.

1.2 Proposed Solution

SmartCare connects a quality-aware acquisition device to a centralized monitoring service, a patient PWA and a separate supervision PWA. The MAX30102 supplies optical readings requiring a documented algorithm to estimate HR and SpO_2 [4]. The backend manages permissions, history and alert episodes; a Python service supplies experimental anomaly scores.

The ambition is a traceable sensor-to-alert workflow. Initial trials use short supervised sessions with a correctly positioned finger sensor; continuous unattended wearable use is not assumed.

Conclusion

SmartCare prioritizes usable monitoring, reliable event handling and an accountable model lifecycle within an academic prototype.

Chapter 2

Customers, Needs and Objectives

Introduction

The actors, usage scenarios and measurable objectives establish who the solution serves and how the first release will be assessed.

2.1 Users and Customers

— **Monitored individual:** personal values, history, notifications and device freshness through the front-office; own records only.

— **Monitoring professional:** assigned-patient comparison, episode review, alert acknowledgement and permitted rule configuration through the back-office.

— **Administrator:** accounts, roles, assignments, devices and service health; no physiological-history access by default.

— **ML maintainer:** approved pseudonymized snapshots, experiments and controlled releases through protected technical tooling.

— **Customer organization:** a potential home-monitoring structure, supervised residence, practice or research/educational organization purchasing a shared tool.

The ML maintainer is a technical responsibility rather than a third PWA. Administrators control enrollment and assignments; the backend resolves each device's patient identity.

2.2 Usage Scenarios

A valid session updates both authorized interfaces and the stored history. A persistent deviation opens one reviewable episode; unreliable contact or connection loss produces a technical state. A model-release exercise demonstrates version tracing and rollback without losing observations.

2.3 Measurable Objectives

O1. Live display: target p95 acquisition-to-browser delay at most 5 s on a controlled network, with synchronized clocks, one derived message/s/device and ten simulated devices. Record sample counts and environment.

O2. Persistent alerts: one episode per scripted qualifying deviation; isolated spikes do not trigger an event. Measure processing delay separately from configured persistence.

O3. Authorization: cross-patient, unassigned-professional and administrator-history denial tests pass for REST, exports and WebSocket subscriptions.

O4. Quality/freshness: invalid samples never enter physiological decisions; device silence produces a visible stale state.

O5. History: interval queries and authorized exports match stored records, timestamps and identity.

O6. Two PWAs: independent manifests, identifiers, start URLs and service-worker scopes; installation and responsive checks at 360 px and desktop width.

O7. ML reproducibility: preserve dataset hash, split, features, seed, parameters and dependencies to reproduce reported metrics within a declared tolerance.

O8. ML serving: attach model/feature versions to scores, test invalid-input and timeout fallback, and target p95 inference below 500 ms on the recorded CPU environment.

O9. Model release: block a failing candidate and demonstrate fixed-version deployment, audit and rollback.

O10. Delivery evidence: link sources, both PWA builds, installation instructions, tests, model card and communication assets to a release tag.

The network and CPU targets are laboratory targets. They are not service-level guarantees. Load tests use pseudonymous simulated streams; only the physical device is used to assess the acquisition path.

2.4 Project Limits and Priorities

V1 requires HR/estimated SpO_2, history, two PWAs, access isolation, persistent rules, an evaluated ML candidate and reproducible deployment/rollback. Motion sensing, Web Push and expanded dashboards are optional. ECG, disease classification, treatment, emergency response, guaranteed 24/7 service and automatic model promotion are excluded. Chip temperature is not claimed as body temperature.

Conclusion

The core release combines monitoring, access isolation and controlled ML operation. Optional features are deferred before these acceptance objectives are reduced.

Chapter 3

Functional Scope

Introduction

This chapter separates the two PWA products and defines shared event behavior.

3.1 Front-Office PWA for the Monitored Individual

The patient application offers login/logout and session expiry (FO-01); current HR/estimated SpO_2, units, time and quality (FO-02); live updates with reconnection status (FO-03); interval charts showing gaps (FO-04); paginated history and permitted CSV export (FO-05); personal episodes/notifications (FO-06); device freshness (FO-07); permitted profile/device data (FO-08); and independent installation (FO-09).

The backend derives patient identity from the session for all reads, exports and subscriptions. Patients cannot change alert policies, review events for professionals or choose model versions. Acceptance checks live updates without reload, matching history and visible stale data. The UI supports responsive layouts, labelled inputs, keyboard navigation and status cues beyond color. Only static assets are cached offline; authenticated physiological data is excluded. Logout clears local application state and ends the session.

3.2 Back-Office PWA for Professionals and Administrators

The back-office provides role-aware authentication (BO-01); assigned-patient list (BO-02); detail/history/export (BO-03); alert evidence, acknowledgement and notes (BO-04); permitted rule configuration (BO-05); operational health (BO-06); accounts/roles/assignments (BO-07); device enrollment/revocation (BO-08); independent installation (BO-09); and model/evaluation status for authorized technical users (BO-10).

Professionals see assigned patients only. Administrators manage technical metadata without physiological-history access by default. Acknowledgement records actor, time and note; policy changes preserve a version. Account or assignment changes revoke affected API

and live-subscription access. Tests demonstrate both permitted operations and rejected access.

3.3 Shared Functions and Alert Lifecycle

The backend alone creates authoritative alerts. Each episode stores source (rule, ML or technical), reason, evidence interval and configuration/model versions. Initial laboratory settings are one derived measurement/s, heartbeat every 10 s and stale status after 30 s. Rule opening requires ten consecutive valid out-of-range observations; invalid or missing data resets the pending sequence.

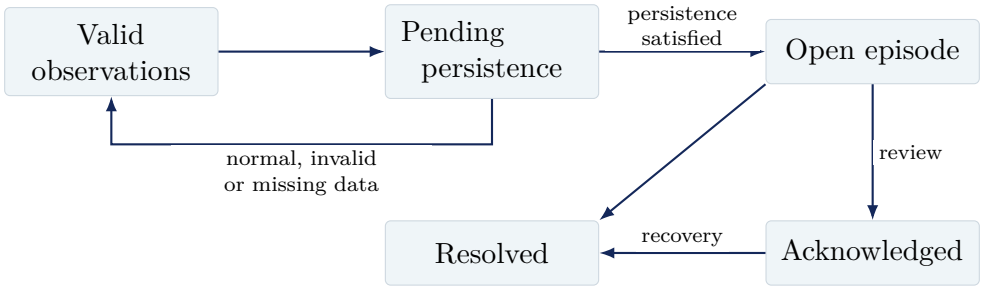


FIGURE 3.1 – Episode lifecycle. Recovery and acknowledgement are separate facts; an open episode can recover before acknowledgement.

Recovery requires ten valid in-range observations. A 60 s cooldown updates an existing episode rather than duplicating it. Acknowledgement and recovery are separate facts. Manual resolution requires an authorized actor and reason. ML persistence is specified in Chapter 5.

3.4 Interface Boundaries

Both PWAs use the same server-side permission checks for REST, exports and live events. Errors avoid revealing another patient's identity. Technical failures, physiological rule episodes and experimental ML events remain distinguishable.

Conclusion

The two applications provide role-specific functions while the backend owns access decisions and the persistent alert lifecycle.

Chapter 4

Business Model

Introduction

This chapter defines the business hypotheses and marketing policy.

4.1 Value Proposition and Customers

The B2B2C hypothesis offers the kit, both interfaces, history and analysis to home-monitoring organizations, supervised residences, practices and research/educational structures. These are prospective customers.

4.2 Revenue and Market Validation

Revenues may combine kit sale/rental, subscriptions, onboarding and support. Pricing must cover hardware, hosting, maintenance and replacements and be tested against willingness to pay. Physiological data is not sold.

Use 3–5 interviews, a demonstration and comparison with manual logs, consumer-device apps and IoT dashboards to test adoption. No market-size or superiority claim is assumed.

4.3 Marketing Mix and Commercialization

The proposed 4P policy is: **Product**, kit/two PWAs/history/alerts/onboarding; **Price**, transparent kit and service costs with explicit support terms; **Place**, direct organizational contact and browser/PWA distribution; **Promotion**, demo video, landing page, brochure, press fact sheet and sample campaign.

After the week-6 academic demonstration, validate demand and a small pilot before deciding on commercial launch.

4.4 Business Model Canvas

Figure 4.1 consolidates these hypotheses using the nine-block framework [11]. Partners, channels and revenues remain to be validated.

Key partners

Potential electronics suppliers

Cloud infrastructure provider

SUP'COM teaching and technical support

Potential supervised pilot organization

Professional reviewer for future use validation

Key activities

Device integration and quality checks
Two-PWA development
Data handling and support
Model evaluation and controlled updates

Key resources

ESP32 and optical sensor
Versioned source and models
Approved datasets
Cloud VM and protected storage
Engineering and supervision time

Value propositions

Centralized personal measurements and trends

Separate patient and supervision experiences

Quality-aware persistent alerts

Reproducible anomaly-analysis lifecycle

Transparent device and model status

Customer relationships

Assisted enrollment
Short user training
Support and maintenance
Regular feedback
Clear review responsibilities

Channels

Direct B2B outreach
University demonstrations
Landing page and video
Browser and installable PWAs
Potential referrals after a pilot

Customer segments

Potential buyers
Home-monitoring structures
Supervised residences
Practices and research/teaching organizations

Users
Monitored individuals
Assigned professionals
Technical administrators

Cost structure

Sensor kits and replacements; cloud hosting and storage; software engineering; model evaluation; security and backups; onboarding, support and communication.

Revenue streams

Kit sale or rental; recurring platform subscription; onboarding and agreed support. Pricing and willingness to pay remain unvalidated.

FIGURE 4.1 – SmartCare Business Model Canvas. Proposed business hypotheses, October 2026. Original SmartCare content arranged using the nine-block framework of Osterwalder and Strategyzer.

Conclusion

The business direction is an organization-funded monitoring service. A pilot must validate both customer value and sustainable operating costs.

Chapter 5

Technical Architecture and Deployment

Introduction

This chapter defines the data path, deployment, model experiment and MLOps implementation. Monitoring operates independently of the training workflow.

5.1 Overview and Technology Choices

Acquisition, integration, application services and presentation form the operational path. ML scoring is asynchronous: an unavailable model must not block storage, display or physiological rules.

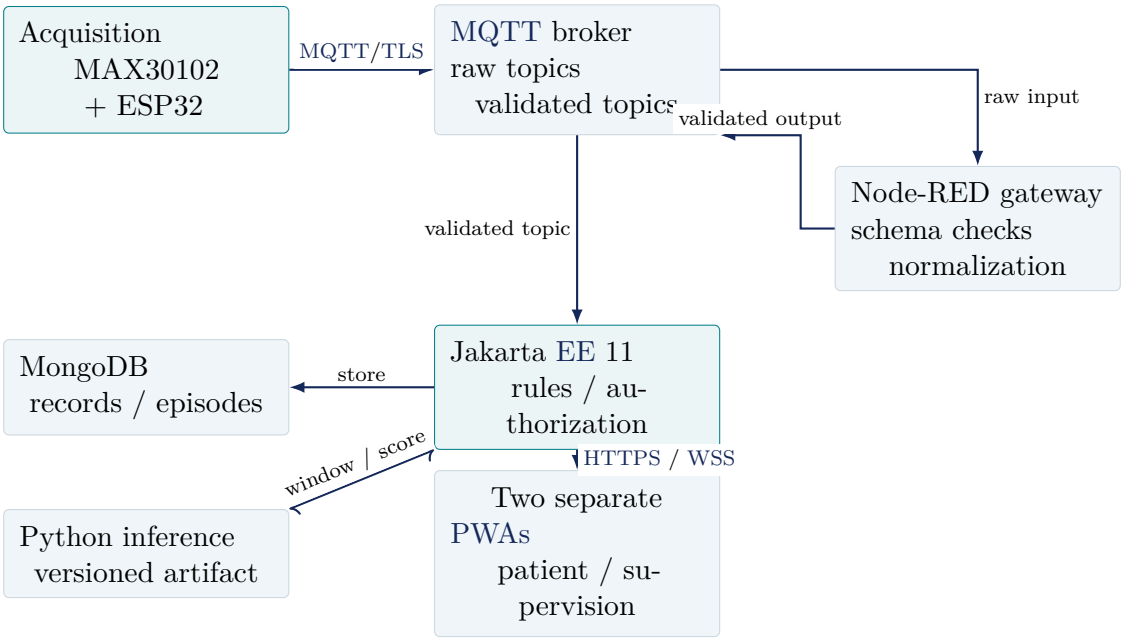


FIGURE 5.1 – Functional architecture. Node-RED uses distinct input/output topics; the backend remains the authority for ownership, storage and alert decisions.

TABLE 5.1 – Core technologies and responsibilities

Technology	**Responsibility**
ESP32 + MAX30102	Optical acquisition, HR/SpO_2 estimates, timestamps and quality.
Mosquitto / MQTT	Authenticated telemetry, TLS, per-device topic ACLs and QoS 1.
Node-RED	Envelope validation, normalization and separate validated-data topics.
Jakarta EE 11	REST, authorization, persistence, rules and WebSocket updates.
MongoDB	Operational users, devices, measurements, scores, alerts and audit.
Python / scikit-learn	Quality/features, baseline and Isolation Forest.
FastAPI	Private, versioned inference API and health checks.
Git / DVC / MLflow	Source/data lineage, experiments and model registry.
Docker Compose	Repeatable deployment, private networks and persistent volumes.

Check Java 21 and a Jakarta EE 11-compatible GlassFish release in week 1; the platform requires Java 17 or above [3]. MLflow uses SQLite metadata and an artifact volume, separately from MongoDB. The kit also needs wires, USB power and basic protection. No GPU or extra physical gateway is required. Freeze tested dependencies/image digests in the release manifest.

5.2 Data Flow and Communication Contracts

The ESP32 derives one summarized message/s from optical samples and publishes to `smartcare/v1/raw/<deviceId>`. Node-RED validates the envelope and units, then republishes to `smartcare/v1/validated/<deviceId>`. Distinct topics prevent a routing loop. The backend rechecks identity, schema and enrollment, stores idempotently, evaluates rules and requests eligible ML-window scores. Authorized PWAs receive REST/WebSocket updates.

QoS 1 permits redelivery [1, 2]. A unique `deviceId/bootId/sequence` key prevents duplicates. Each message includes schema version, UTC acquisition time, HR, estimated SpO_2, quality/reason, clock status and source (physical, replay or simulated). Reception time is recorded separately. Invalid estimates use null plus a reason; stale replays preserve their original time.

MongoDB stores accounts/assignments, device-enrollment history, measurements, baselines, scores, alert episodes and configuration/audit records. Index patient/time history, unique message keys and open episodes. Scores retain the window, model and feature versions; episodes retain evidence and review history. Historical enrollment prevents delayed data from being assigned to a newly enrolled individual.

5.3 Separate PWAs, Security and Reliability

Deploy independent builds under `/patient/` and `/supervision/`, each with its own manifest, app ID, start URL and non-overlapping service-worker scope [12]. The shared API and event paths are outside both cache scopes. Sessions use Secure/HttpOnly cookies, an appropriate SameSite policy and CSRF protection. The server checks ownership and assignment for REST and every live subscription, including after revocation.

Only necessary HTTPS/WSS and MQTT/TLS listeners are exposed. MongoDB, inference, Node-RED administration and MLflow remain private. Device ACLs, service identities and protected secrets restrict access. Bounded device replay handles short outages; stale-state indicators expose interruptions. Inference uses a proposed 1 s timeout and bounded queue. Invalid input, timeout or model mismatch produces an explicit unavailable state while rules continue. Backups and restore instructions accompany deployment.

5.4 Cloud Deployment and UML Deployment Diagram

The prototype uses one Linux IaaS VM with Docker Compose and persistent volumes; this topology has a single point of failure. An initial 2–4 vCPU/4–8 GB RAM allocation is a capacity hypothesis to benchmark. MLflow/training belong to the protected management plane. Serving loads one fixed artifact and does not depend on tracking-server availability.

Cloud code is retrieved through GitHub clone/pull to a known release commit, as required by the course. Protected data/artifacts are retrieved separately by version/checksum. The release manifest links them to source and dependency versions; each pushed revision remains compilable.

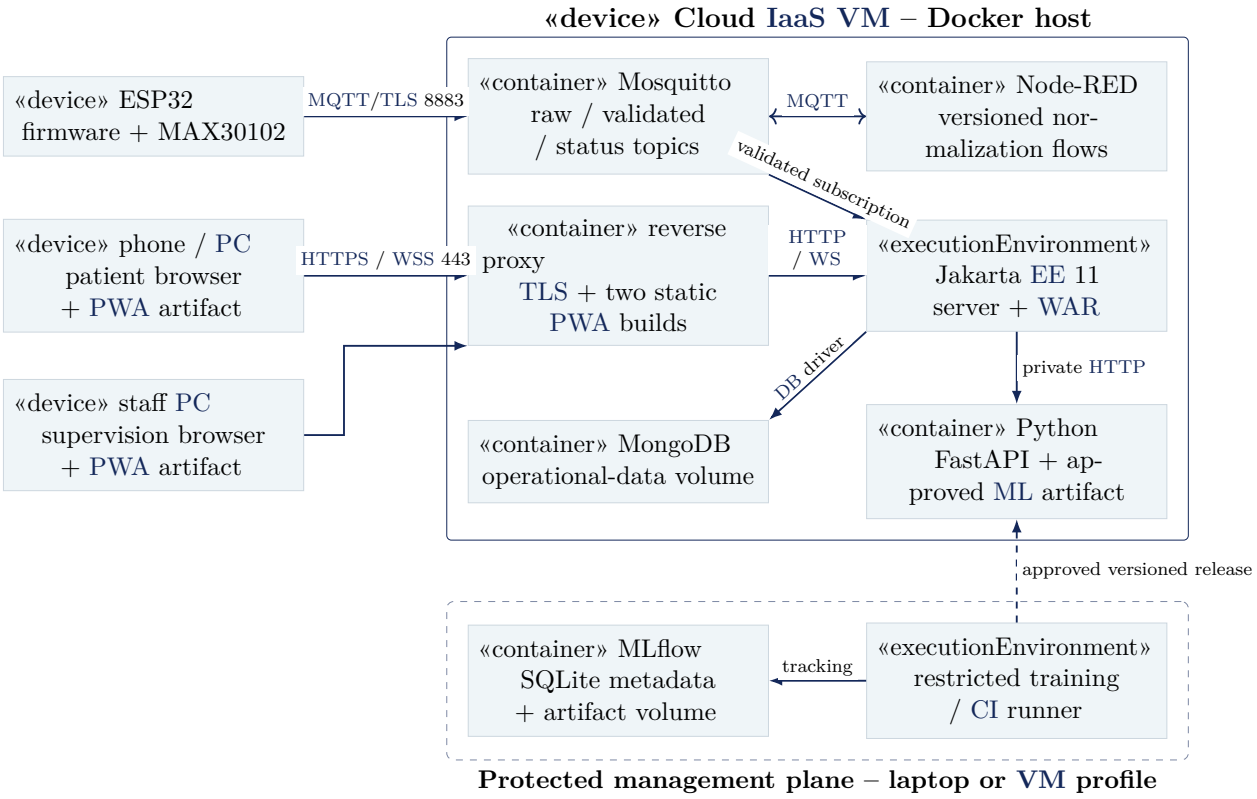


FIGURE 5.2 – UML deployment view of SmartCare. Internal services are not public endpoints; volumes persist outside container lifetimes. HTTP on the private host network requires network isolation; use TLS if services cross hosts.

5.5 Machine Learning Integration and Model Development

5.5.1 Learning Task and Data

The task is anomaly detection on valid HR/SpO_2 windows. Isolation Forest is compared with persistence rules and a robust deviation baseline [5]. Its score measures unusualness, not disease probability.

Collect short approved stationary-finger sessions with source, quality and session identifiers. A feasibility target is ten independent 10–15 minute sessions across several days; this does not establish clinical generalization. Seeded synthetic sessions cover nominal values, sustained shifts, isolated spikes, missing data, duplicates and disconnects. Physical stability/false-event results and synthetic injected-event results are reported separately. Public datasets are optional after license, unit and feature checks. A review acknowledgement is not automatically a training label.

5.5.2 Quality and Feature Schema

Use trailing 60 s windows every 10 s at the proposed 1 Hz message rate. Eligibility requires at least 80% valid observations and no gap over 5 s. Compute HR median/IQR/range, SpO_2 median/IQR/minimum, slopes and deviations from prior personal reference statistics. Quality metrics govern eligibility rather than physiological scoring. Movement features require a separately versioned schema.

For a reference median $\mu_{p,j}$ and median absolute deviation $\mathrm{MAD}_{p,j}$,

$$z_{p,j} = \frac{x_j - \mu_{p,j}}{\max(1.4826\,\mathrm{MAD}_{p,j}, \epsilon_j)}.$$

Record each scale floor ϵ_j and calibration interval. Personal references use earlier eligible data only; an initial minimum is 30 reference windows from two sessions. Until sufficient reference data exists, display that limitation and retain rules. Variability of 1 Hz HR estimates is not claimed as ECG-derived HRV.

5.5.3 Training, Validation and Model Output

Split independent sessions approximately 60/20/20 into training, validation and locked test *before* generating overlapping windows. Fit transformations and the estimator on training only, tune on validation and evaluate the final snapshot once [6]. Synthetic partitions use different seeds and a held-out scenario/magnitude range. If enough participants exist, add leave-person-out analysis; do not claim it from one participant.

Begin with one pooled forest rather than a model per patient. Compare 100/200 trees and `max_samples=128/256` where data permits; start with `max_features=1.0`, `bootstrap=False`, seed 42 and `contamination='auto'`. Choose a separate validation threshold τ for $a(x) = -\text{score_samples}(x)$, so larger scores mean more unusual windows. Return score, threshold, eligibility/reason, window interval and model/feature versions. Bundle transformations, ordered features, threshold and dependencies with the estimator.

5.5.4 Evaluation and Alert Integration

Report labelled-window precision/recall/F1/PR-AUC where applicable, event recall, false episodes per nominal hour, detection delay, counts and p95 inference latency. Compare with the baseline at the same false-event budget. Proposed engineering gates are synthetic event recall ≥ 0.90, at most one false episode per nominal simulated hour and p95 inference ≤ 500 ms on the recorded CPU environment. These are targets, not achieved accuracy or clinical evidence.

The backend sends candidate windows to private `POST /predict`; Python uses the same eligibility/feature library as offline training. Initial ML persistence is three consecutive eligible anomalous windows. Report detection delay including windowing/persistence. ML never suppresses a rule alert. A candidate with insufficient or failed evidence stays in shadow mode without patient-facing ML notifications.

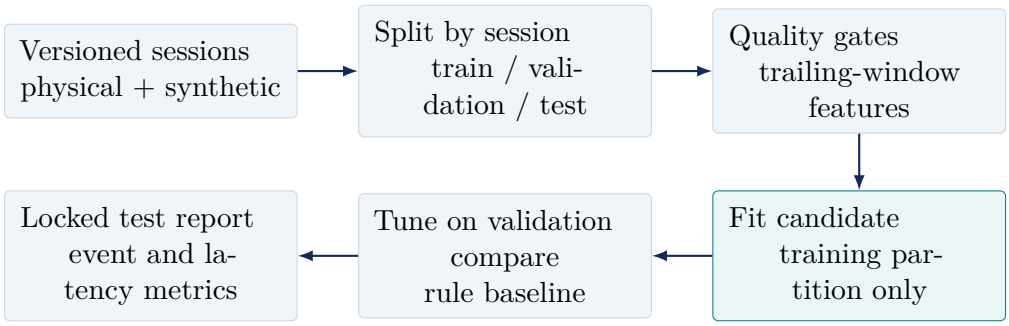


FIGURE 5.3 – Experimental ML pipeline. Session boundaries are fixed before overlapping windows are generated.

5.6 MLOps Implementation

5.6.1 Versioning, Tracking and Registry

The reproducibility key combines Git commit, DVC dataset hash, split manifest, feature version, seed, parameters and dependency lock. Commit DVC metadata; keep physiological snapshots in approved private storage and synthetic fixtures in public CI [7]. MLflow records parameters, metrics and evaluation artifacts [8]. Registered candidates link to a run/model card. Resolve a reviewed alias to a fixed version and checksum; alias changes must not silently change serving [9].

5.6.2 CI, Deployment and Rollback

1. On code changes, run schema/unit/feature tests and both independent PWA builds using synthetic fixtures.
2. An authorized training run loads the approved snapshot, computes features, fits the candidate and logs the experiment.
3. Validate baseline comparison, metrics, signature, dependencies, artifact integrity and timeout/fallback behavior.
4. A second member reviews the model card and release record; failed/inconclusive candidates remain in shadow mode.
5. Deploy an immutable model/image version, then run health, smoke and end-to-end checks.
6. Roll back to the previous approved version, verify a known input and retain the failed release/reason in history.

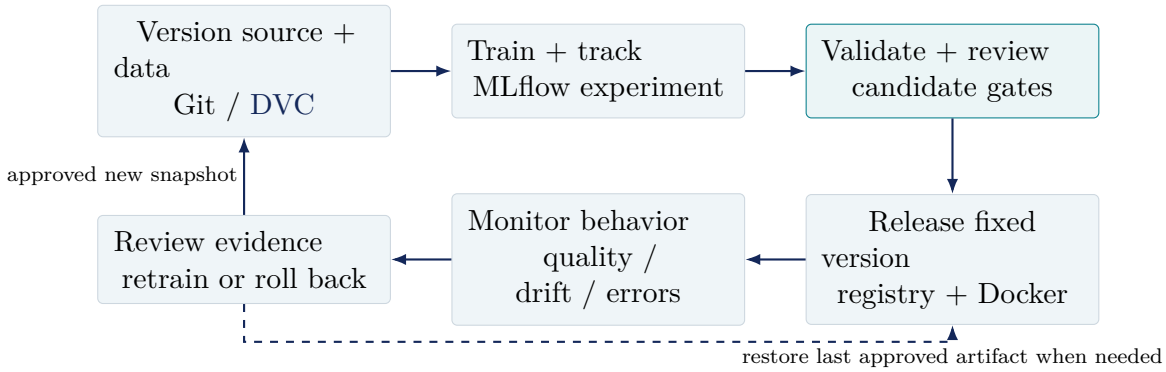


FIGURE 5.4 – MLOps lifecycle with explicit review and rollback. Data drift requests investigation; it does not authorize automatic deployment.

5.6.3 Monitoring and Drift Review

Track ingestion/rejection/duplicate counts, stale devices, invalid ratios, inference latency/errors, missing scores, active version and alert rate. A protected metrics endpoint plus a back-office summary is sufficient; Grafana/Prometheus is optional.

Compare recent feature distributions with fixed training-reference bins using population stability index (PSI). An initial review heuristic is PSI above 0.2 after 200 eligible windows in two successive batches. Record bins, smoothing and batch definitions; overlap and small samples limit interpretation. Also inspect quality, missingness and score distributions. Drift signals investigation rather than proven accuracy loss or automatic retraining. New snapshots require review and the same release gates.

5.6.4 Model Card and Serving Contract

The model card states use/exclusions, data provenance/counts, split, quality rules, features, parameters, score convention, thresholds, metrics, limitations, maintainer and rollback version. Private `/predict`, `/health` and `/model-info` expose readiness and loaded-version metadata. Feature mismatch/timeout returns an unavailable or invalid reason, never a fabricated normal score. Without reliable delayed labels, live precision/recall cannot be claimed.

5.7 Standard IoT Models, WoT and Design Patterns

Thing/Sensor/Payload represent the device, adapter and observation; BrokerClient/IoT-Protocol encapsulate MQTT; Service/Property expose measurements; HumanActor/Role/Permission express assignments; Command/Invoker/Receiver support audited actions. No treatment actuator is required.

The virtual device has a versioned W3C WoT Thing Description [13]: latest-measurement/connection properties, update events, JSON schemas, security requirements and forms pointing to implemented endpoints. It contains no credentials and grants no permission by itself.

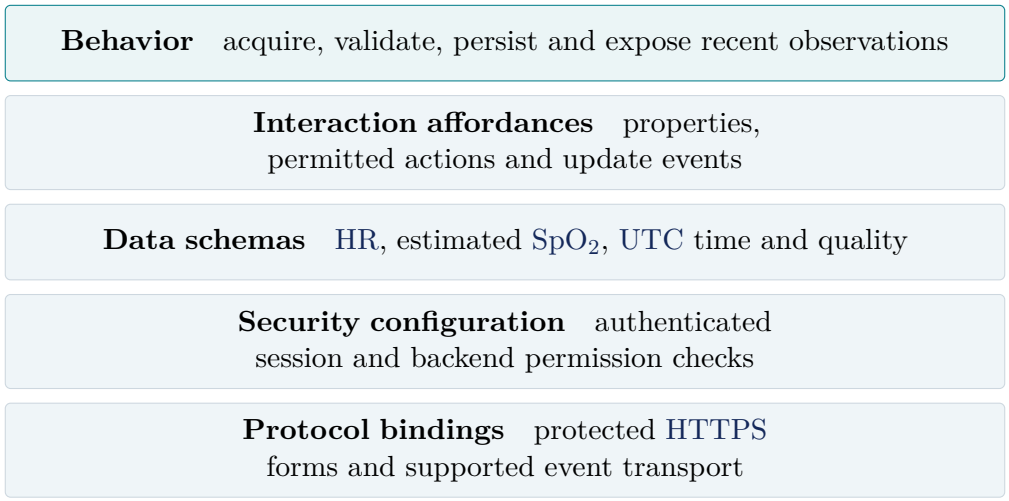


FIGURE 5.5 – Conceptual WoT layering for SmartCare. The Thing Description describes the exposed interactions; the implementation supplies behavior and enforces authorization.

Use Observer for updates, Strategy for scorers, Chain of Responsibility for validation and Command for audited operations. The design notebook refines the class/sequence diagrams and justifies each pattern rather than introducing unused abstractions.

Conclusion

The design specifies an operational path that survives ML failure and an experiment/release path that can be reproduced, reviewed and rolled back.

Chapter 6

Deliverables and Validation Criteria

Introduction

Deliverables are defined by reviewable artifacts and acceptance evidence for the scope and development phases.

6.1 Deliverables of the Scope Statement Phase

The GitHub submission due on 24 October 2026 at 23:59 UTC+1 comprises this PDF, its compilable LaTeX source and a README, accompanied by the presentation video required by the course. The document includes the eight scope chapters, BMC, functional and deployment views, model/MLOps specifications, Gantt, costs and effort assumptions.

6.2 Deliverables of the Development Phase

D1. IoT prototype: firmware, wiring, estimation-library version and timestamped quality-aware acquisition.

D2. Broker/gateway: TLS/ACL setup, versioned Node-RED flows, validation and reconnect evidence.

D3. Backend: Jakarta EE sources, API contracts, MongoDB collections/indexes and authorization/alert tests.

D4. Front-office: independently installable patient PWA with dashboard, history, notifications and offline state.

D5. Back-office: independently installable supervision/admin PWA with assignments and audited review.

D6. Data/features: provenance manifest, approved private snapshot reference, synthetic generator, split and feature schema.

D7. Model: reproducible training/evaluation, baseline comparison, counts, metrics and model card.

D8. MLOps: DVC metadata, MLflow run/registry evidence, CI workflow, fixed-version serving and rollback test.

D9. Deployment: Compose files, configuration template, dependency manifest, health checks and restore instructions.

D10. Technical documentation: architecture, API, security, installation and troubleshooting.

D11. User manual: patient, professional and admin instructions, including quality/freshness interpretation.

D12. Test report: unit/integration, authorization, latency, connectivity and model-gate results with conditions and failures.

D13. Demonstration/marketing: script, presentation, video, landing-page content and brochure.

D14. Press/campaign: short press fact sheet and sample advertising assets; actual paid publication is excluded.

D15. Conceptual notebook: second course deliverable with use cases, sequence/communication, component, deployment, package/class, activity, state and timing diagrams; justify IoT abstractions and patterns.

Deliver source under an explicit academic license choice while respecting dependencies. Exclude private data and secrets; marketing outputs are draft assets.

Each deliverable includes a narrated video of at most five minutes, with project/deliverable/team/date identification, clear voice and no music. Provide a Postman cloud-API collection. Each member tests both PWAs on a supported phone and records installation limits. Organize sources, flows and evidence in GitHub.

6.3 Validation and Traceability

The release checklist links **O1/O4/O5** to acquisition, quality and history evidence; **O2** to the episode lifecycle; **O3** to REST/WebSocket/export denial tests; **O6** to two installation records; **O7/O8** to the reproduced run, held-out evaluation and fallback; **O9** to deployment/rollback; and **O10** to the complete tagged release.

The final demo covers acquisition, both PWAs, a persistent episode and its review, bad signal/disconnection, denied access, a tracked model run and deployment/rollback. Failed metrics remain visible and justify shadow mode.

Conclusion

Acceptance requires working components, reproducible model evidence, documentation and demonstrable role boundaries. Sources and tests remain linked to the same release.

Chapter 7

Constraints and Methodology

Introduction

The project balances time, budget and scope through short XP iterations and explicit quality controls.

7.1 Technological and Hardware Constraints

Jakarta EE 11, MQTT, Node-RED, two PWAs, course IoT abstractions and GitHub-to-cloud deployment are retained. MongoDB and Python are team choices. Verify compatibility through an early end-to-end spike. Sensor quality depends on contact, motion, wiring and the estimation library; the prototype promises no battery life, wearable comfort or uninterrupted acquisition. Bounded replay cannot recover an arbitrary outage.

7.2 Time, Budget and Scope Control

The six-week baseline totals 61 person-days (Chapter 8). Defer extra sensors, deep models, automatic retraining and expanded dashboards before core integration and access isolation. The provisional core cash envelope is **190 TND**: ESP32 35, MAX30102 45, accessories 20, power/cable 15, enclosure 20, hosting 30 and contingency 25. Optional motion sensing adds 20 TND. These are internal allowances requiring quotations. At an illustrative 100 TND/person-day, labor is valued at 6,100 TND, separately from cash spending.

7.3 Security, Privacy and Retention

Prefer synthetic identities and collect only necessary fields. Real sessions need explicit permission and an agreed handling protocol. Separate identity mappings from restricted pseudonymized ML snapshots. Propose 30-day operational retention and deletion/approved archival within 30 days of assessment, subject to supervisor/participant confirmation. Specify deletion across exports, backups and future snapshots. Pseudonymization and TLS alone do

not establish anonymity or regulatory certification.

7.4 eXtreme Programming Methodology

Use a prioritized backlog, weekly demonstration, brief daily coordination, rotating pairs, collective review and simple designs [16]. A story is done when reviewed code passes checks, is documented and demonstrates its acceptance example. Unit tests cover transformations/state changes; contracts cover messages/APIs; integration tests cover ingestion, PWAs and fallback.

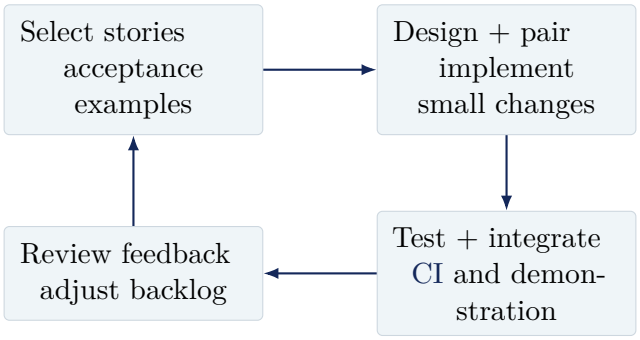


FIGURE 7.1 – Weekly XP iteration. Integration, testing and feedback accompany development.

Source headers identify module, team, contributor, creation/modification and version. Document meaningful methods and HTML/CSS blocks; keep diagrams and implementation consistent. Commit small compilable increments.

7.5 Risks and Preventive Measures

Test acquisition early and keep a simulator for hardware failure. Separate topics and deduplicate ingestion. Exercise assignment revocation and cache restrictions. Insufficient ML data keeps the candidate in shadow mode; a failed artifact returns serving to the last approved version. Health/freshness indicators, backups and a local Compose demonstration address cloud outages. Weekly re-estimation controls scope overrun.

Conclusion

Early integration, meaningful tests and weekly scope review address the main risks while preserving privacy, quality awareness and reproducible delivery.

Chapter 8

Planning and Effort Estimation

Introduction

This chapter adopts the six-week planning baseline of the initial scope statement, with eleven work packages and a total effort of 61 person-days.

8.1 Planning Assumptions

The realization period is six weeks, to be confirmed against the course calendar. Work follows short XP iterations and milestone demonstrations. In Table 8.1, R1, R2 and R3 denote the three contributors; mapping these resource codes to names is confirmed by the team. They are distinct from milestones M1–M4.

8.2 Tasks, Dependencies, and Effort

Effort is expressed in person-days rather than calendar duration. The 61-day baseline amounts to roughly 20 days per contributor, or 3.4 productive days per week each. The allocations are R1: 21, R2: 20 and R3: 20 days. Shared design accounts for two days per contributor.

TABLE 8.1 – Task breakdown and effort estimation

ID	Task	Lead	After	Days	Deliverable	Gate
T1	Design, API contracts, data model	All	–	6	Scope, API contracts, UML	M1
T2	IoT acquisition (ESP32, MAX30102)	R3	T1	6	Timestamped-measurement firmware	M2
T3	Node-RED gateway and MQTT	R3	T1	3	Gateway flows, broker setup	M2
T4	Jakarta EE 11 backend	R2	T1	10	Authentication, REST, alerts, live updates	M2
T5	MongoDB schema/storage	R2	T1	3	Collections and indexes	M2
T6	Front-office PWA	R1	T4	6	Patient app, FO-01–09	M3
T7	Back-office PWA	R1	T4	8	Supervision/admin app, BO-01–10	M3
T8	Python anomaly service	R3	T5	5	Features, model and scores	M3
T9	Integration/cloud deployment	R2	T4, T6, T7	5	Deployed, reproducible services	M4
T10	Unit, integration, acceptance tests	R1	T4,T9	5	Test/evaluation report	M4
T11	Documentation/marketing	R3	T1	4	Manuals, brochure, video	M4
			Total	**61**	R1: 21; R2: 20; R3: 20	

To cover the model lifecycle without adding new task rows, T8 includes the baseline, Isolation Forest experiment and model card; T9 includes DVC/MLflow setup, fixed-version serving and rollback; T10 verifies model gates and fallback. These activities consume the stated allocations and must be re-estimated after the first spike. UI work can begin with T1 contracts and mocks while T4 is implemented; listed dependencies describe integration prerequisites. Tests and documentation are updated incrementally.

8.3 Milestones

— **M1, end of week 1:** validated design, frozen API contracts and data model; initial compatibility and CI checks.

— **M2, end of week 3:** end-to-end acquisition, gateway, backend and storage flow.

— **M3, middle of week 5:** both PWAs functional, with the analysis service integrated and its evaluation recorded.

— **M4, end of week 6:** integrated/tested/documented platform and final demonstration, including model fallback and rollback.

8.4 Gantt Chart

Figure 8.1 retains the original eleven-task calendar. Bars show scheduling windows rather than continuous full-time assignments. M2 and M3 are integration gates; backend and UI refinements continue within their task windows.

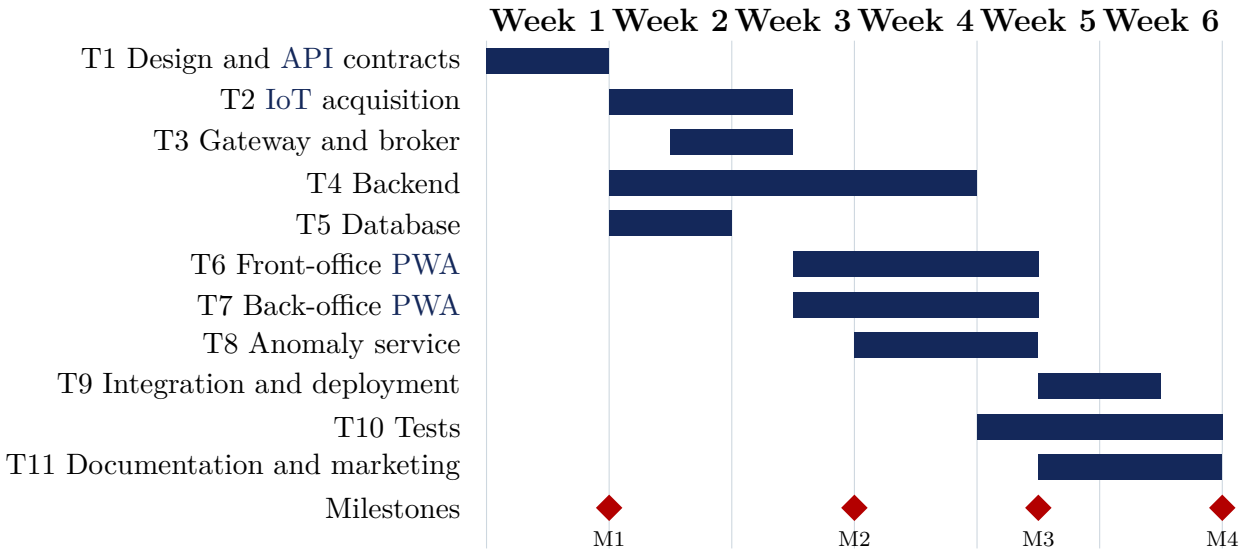


FIGURE 8.1 – Project Gantt chart over six weeks, following the initial scope statement.

8.5 Effort Estimation Method

The task estimate is based on expert judgement of the functional scope in Chapter 3. It must be cross-checked using the course's CoCoMo+ worksheet: estimate software size (function points or delivered source lines), identify cost drivers, apply the prescribed coefficients and compare the result with 61 person-days [14]. The supplied brief does not specify the exact CoCoMo+ calibration; no completed calculation is claimed here.

Record actual time each week and re-estimate remaining work. There is no separately allocated contingency in the original 61-day total. A significant effort gap first removes optional activity sensing, Web Push and expanded dashboards; core PWAs, rule alerts and minimum reproducible ML/MLOps evidence remain required. If the core still exceeds capacity, revise the scope with the supervisor. Draft press and advertising assets remain modest deliverables rather than a live campaign.

Conclusion

The original six-week, 61-person-day baseline is retained, with explicit integration gates and the ML/MLOps work located within the existing tasks. Weekly evidence determines whether the scope remains feasible.

Conclusion

SmartCare is specified as a traceable Cloud of Things prototype joining physiological acquisition, quality-aware processing, two independent PWAs and experimental anomaly analysis. Its eight chapters define the concept, actors, measurable objectives, functional boundaries, business hypotheses, architecture, deliverables, constraints and six-week planning baseline.

The report preserves the first scope's 61-person-day estimate while making model development and MLOps concrete: versioned sessions, leakage-controlled evaluation, registered artifacts, reviewed releases, monitoring and rollback. The remaining commitments are to verify hardware/software compatibility, freeze contracts, validate the data protocol and reconcile effort with the course estimation model. The final demonstration must present measured evidence and limitations alongside the working system.

Bibliography

[1] OASIS. *MQTT Version 5.0*, 2019. Official standard.

[2] Eclipse Paho. *MQTT Client Quality of Service*. Client documentation.

[3] Eclipse Foundation. *Jakarta EE Platform 11*. Platform specification.

[4] Analog Devices. *MAX30102 High-Sensitivity Pulse Oximeter and Heart-Rate Sensor*. Datasheet.

[5] scikit-learn. *IsolationForest: parameters, scores and references*. API documentation.

[6] scikit-learn. *Common pitfalls and recommended practices*. Evaluation guidance.

[7] Iterative. *Data Version Control User Guide*. DVC documentation.

[8] MLflow. *Experiment Tracking*. Tracking documentation.

[9] MLflow. *Model Registry Workflows*. Registry documentation.

[10] MLflow. *Backend Stores*. Storage documentation.

[11] A. Osterwalder / Strategyzer. *Business models: the nine building blocks*. Business Model Canvas framework.

[12] MDN Web Docs. *Progressive Web Apps*. PWA documentation.

[13] W3C. *Web of Things Thing Description 1.1*, Recommendation, 2023. W3C specification.

[14] B. W. Boehm. *Software Engineering Economics*. Prentice-Hall, 1981.

[15] M.-B. Kaâniche. *Cloud of Things Project Guidelines*, supplied course slides, scope update 8 October 2026.

[16] K. Beck and C. Andres. *Extreme Programming Explained: Embrace Change*, 2nd ed. Addison-Wesley, 2004.